

TUGAS BESAR 1

IF3270 PEMBELAJARAN MESIN

FEEDFORWARD NEURAL NETWORK



Oleh
Nama Kelompok: trizzzcuIt

Rafif Farras 13523095

Barru Adi Utomo 13523101

Azfa Radhiyya Hakim 13523115

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
PROGRAM STUDI TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

DESKRIPSI PERSOALAN

Tugas Besar I pada kuliah IF3270 Pembelajaran Mesin agar peserta kuliah mendapatkan wawasan tentang bagaimana cara mengimplementasikan Feedforward Neural Network (FFNN). Pada tugas ini, peserta kuliah akan ditugaskan untuk mengimplementasikan FFNN from scratch.

Implementasikan suatu modul FFNN yang memenuhi ketentuan-ketentuan berikut:

- FFNN yang diimplementasikan dapat **menerima jumlah neuron dari tiap layer** (termasuk input layer dan output layer).
- FFNN yang diimplementasikan dapat **menerima fungsi aktivasi dari tiap layer**. Pilihan fungsi aktivasi yang harus diimplementasikan adalah sebagai berikut:

Nama Fungsi Aktivasi	Definisi Fungsi
Linear	$Linear(x) = x$
ReLU	$ReLU(x) = \max(0, x)$
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$
Hyperbolic Tangent (tanh)	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
Softmax	Untuk vector $\vec{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, $softmax(\vec{x})_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$

- FFNN yang diimplementasikan dapat **menerima fungsi loss** dari model tersebut. Pilihan loss function yang harus diimplementasikan adalah sebagai berikut:

Nama Fungsi Aktivasi	Definisi Fungsi
MSE	$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
Binary Cross-Entropy	$\mathcal{L}_{BCE} = -\frac{1}{n} \sum_{i=1}^n (y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i))$ y_i = Actual binary label (0 or 1) $\hat{y}_i$ = Predicted value of y_i n = Batch size
Categorical Cross-Entropy	$\mathcal{L}_{CCE} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^C (y_{ij} \log \hat{y}_{ij})$ y_{ij} = Actual value of instance i for class j $\hat{y}_{ij}$ = Predicted value of y_{ij} C = Number of classes n = Batch size

- Catatan:
 - Binary cross-entropy merupakan kasus khusus categorical cross-entropy dengan kelas sebanyak 2.
 - Log yang digunakan merupakan logaritma natural (logaritma dengan basis e).
- Terdapat mekanisme untuk **inisialisasi bobot** tiap neuron (termasuk bias). Pilihan metode inisialisasi bobot yang harus diimplementasikan adalah sebagai berikut:
 - **Zero initialization**
 - Random dengan distribusi **uniform**.
 - Menerima parameter lower bound (batas minimal) dan upper bound (batas maksimal).
 - Menerima parameter seed untuk reproducibility.
 - Random dengan distribusi **normal**.
 - Menerima parameter mean dan variance.

- Menerima parameter seed untuk reproducibility.
- Instance model yang diinisialisasikan harus bisa **menyimpan bobot** tiap neuron (termasuk bias).
- Instance model yang diinisialisasikan harus bisa **menyimpan gradien bobot** tiap neuron (termasuk bias).
- Instance model memiliki method untuk **menampilkan distribusi bobot** dari tiap layer.
 - Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot.
- Instance model memiliki method untuk **menampilkan distribusi gradien bobot** dari tiap layer.
 - Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot.
- Instance model memiliki method untuk **save** dan **load**.
- Model memiliki implementasi **forward propagation** dengan ketentuan sebagai berikut:
 - Dapat menerima input berupa **batch**.
- Model memiliki implementasi **backward propagation** untuk menghitung perubahan gradien:
 - Dapat menangani perhitungan perubahan gradien untuk input data **batch**.
 - Gunakan konsep **chain rule** untuk menghitung gradien tiap bobot terhadap loss function.
 - Berikut merupakan **turunan pertama** untuk setiap fungsi aktivasi:

Nama Fungsi Aktivasi	Turunan Pertama
Linear	$\frac{d(\text{Linear}(x))}{dx} = 1$
ReLU	$\frac{d(\text{ReLU}(x))}{dx} = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$
Sigmoid	$\frac{d(\sigma(x))}{dx} = \sigma(x)(1 - \sigma(x))$
Hyperbolic Tangent (tanh)	$\frac{d(\tanh(x))}{dx} = \left(\frac{2}{e^x - e^{-x}} \right)^2$

Softmax	Untuk vector $\vec{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$, $\frac{d(\text{softmax}(\vec{x}))}{d\vec{x}} = \begin{bmatrix} \frac{\partial(\text{softmax}(\vec{x}))_1}{\partial x_1} & \dots & \frac{\partial(\text{softmax}(\vec{x}))_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial(\text{softmax}(\vec{x}))_n}{\partial x_1} & \dots & \frac{\partial(\text{softmax}(\vec{x}))_n}{\partial x_n} \end{bmatrix}$ Dimana untuk $i, j \in \{1, \dots, n\}$, $\frac{\partial(\text{softmax}(\vec{x}))_i}{\partial x_j} = \text{softmax}(\vec{x})_i (\delta_{ij} - \text{softmax}(\vec{x})_j)$ $\delta_{i,j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$
---------	--

- Berikut merupakan **turunan pertama** untuk setiap fungsi loss terhadap bobot suatu FFNN (lanjutkan sisanya menggunakan chain rule):

Nama Fungsi Loss	Definisi Fungsi
MSE	$\frac{\partial \mathcal{L}_{MSE}}{\partial W} = -\frac{2}{n} \sum_{i=1}^n \frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial W} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \frac{\partial \hat{y}_i}{\partial W}$
Binary Cross-Entropy	$\frac{\partial \mathcal{L}_{BCE}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \frac{\partial \mathcal{L}_{BCE}}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)} \frac{\partial \hat{y}_i}{\partial W}$
Categorical Cross-Entropy	$\frac{\partial \mathcal{L}_{CCE}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^c \frac{\partial \mathcal{L}_{CCE}}{\partial \hat{y}_{ij}} \frac{\partial \hat{y}_{ij}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^c \frac{y_{ij}}{\hat{y}_{ij}} \frac{\partial \hat{y}_{ij}}{\partial W}$

- Model memiliki implementasi metode **regularisasi L1** dan **L2**.
- Model memiliki implementasi **weight update** dengan menggunakan **gradient descent** untuk memperbarui bobot berdasarkan gradien yang telah dihitung, berikut persamaannya:

$$W_{new} = W_{old} - \alpha \left(\frac{\partial \mathcal{L}}{\partial W_{old}} \right)$$

α = Learning rate

- Implementasi untuk pelatihan model harus memenuhi ketentuan berikut:
 - Dapat menerima parameter berikut:
 - Batch Size
 - Learning Rate
 - Jumlah Epoch

■ Verbose

- Verbose 0 berarti tidak menampilkan apa-apa selama pelatihan.
- Verbose 1 berarti hanya menampilkan progress bar beserta dengan kondisi training loss dan validation loss saat itu.
 - Proses pelatihan mengembalikan **histori dari proses pelatihan** yang berisi **training loss** dan **validation loss tiap epoch**

BAB 1

PENJELASAN IMPLEMENTASI

1.1 Class FFNN

1.1.1 Definisi

Feed Forward Neural Network (FFNN) merupakan salah satu arsitektur dasar dari jaringan saraf tiruan yang banyak digunakan dalam berbagai permasalahan machine learning seperti klasifikasi maupun regresi. Pada jaringan ini, informasi mengalir secara satu arah, yaitu dari input layer, lanjut ke hidden layer, dan di akhir ke output layer, tanpa adanya siklus.

Pada implementasi ini dibuat sebuah kelas Python bernama FFNN yang berfungsi untuk membangun, melatih, serta mengevaluasi model neural network secara modular. Implementasi ini juga mendukung beberapa komponen penting dalam neural network seperti:

- Berbagai fungsi aktivasi
- Beberapa metode inisialisasi bobot
- Loss function
- Regularisasi
- Mini-batch gradient descent
- Visualisasi distribusi bobot dan gradien

1.1.2. Atribut Kelas

Class FFNN terdiri atas beberapa atribut yang dijelaskan sebagai berikut.

- `layer_sizes`: jumlah neuron pada setiap layer
- `activations`: fungsi aktivasi yang digunakan tiap layer
- `weights`: matriks bobot antar layer
- `biases`: nilai bias tiap layer
- `grad_weights`: gradien bobot hasil backpropagation
- `grad_biases`: gradien bias hasil backpropagation
- `adam_t`: counter timestep update Adam, dan dipakai untuk koreksi bias
- `adam_m_w`, `adam_v_w`: menyimpan first moment dan second moment untuk gradien bobot
- `adam_m_b`, `adam_v_b`: menyimpan first dan second moment untuk gradien bias
- `adam_m_g`, `adam_v_g`: menyimpan first dan second moment untuk gradien gamma RMSNorm

1.1.3. Parameter Kelas

Metode `__init__` digunakan untuk menginisialisasi model FFNN, yang membutuhkan beberapa atribut yang dijabarkan sebagai berikut.

1. `layer_number_list`
2. Menyatakan jumlah neuron pada setiap layer, dimana indeks 0 merupakan input layer dan indeks terakhir merupakan output layer, dan sisanya adalah hidden layer. Sebagai contoh, jika input berupa `[4, 8, 3, 2]`, maka model terdiri dari 2 hidden network.
3. `activation_number_list`
Menyatakan fungsi aktivasi pada setiap layer selain input, sehingga panjang nya wajib bernilai banyak anggota `layer_number_list` dikurangi dengan 1.
4. `weight_initial`

Metode inialisasi bobot yang digunakan. Terdapat 3 pilihan yaitu

- Zero → semua bobot bernilai 0
- Uniform → bobot diambil menggunakan distribusi uniform
- Normal → bobot diambil menggunakan distribusi normal

1.1.4. Metode

Class FFNN dibangun dengan beberapa metode berikut:

1. `__init_weights()`

Digunakan untuk menentukan metode inialisasi bobot yang akan digunakan pada model

```
def __init_weights(self, method: str) -> None:
    if method == "zero":
        self._init_zero()
    elif method == "uniform":
        self._init_uniform(
            lower=-1.0,
            upper=1.0,
            seed=None,
        )
    elif method == "normal":
        self._init_normal(
            mean=0.0,
            variance=1.0,
            seed=None,
        )
```

2. `__init_zero()`

Digunakan untuk menginisialisasi seluruh bobot dengan nilai nol.

```
def __init_zero(self) -> None:
    for i in range(self.num_layers - 1):
        fan_in = self.layer_sizes[i]
        fan_out = self.layer_sizes[i + 1]
        self.weights.append(np.zeros((fan_in, fan_out)))
        self.biases.append(np.zeros((1, fan_out)))
        self.grad_weights.append(np.zeros((fan_in, fan_out)))
        self.grad_biases.append(np.zeros((1, fan_out)))
```

3. `__init_uniform()`

Digunakan untuk menginisialisasi bobot menggunakan distribusi uniform.

```
def __init_uniform(self, lower: float, upper: float, seed: Optional[int])
-> None:
```

```

rng = np.random.default_rng(seed)
for i in range(self.num_layers - 1):
    fan_in = self.layer_sizes[i]
    fan_out = self.layer_sizes[i + 1]
    self.weights.append(rng.uniform(lower, upper, size=(fan_in,
fan_out)))
    self.biases.append(np.zeros((1, fan_out)))
    self.grad_weights.append(np.zeros((fan_in, fan_out)))
    self.grad_biases.append(np.zeros((1, fan_out)))

```

4. `_init_normal()`

Menginisialisasi bobot menggunakan distribusi normal.

```

def _init_normal(self, mean: float, variance: float, seed: Optional[int])
-> None:
    std = np.sqrt(variance)
    rng = np.random.default_rng(seed)
    for i in range(self.num_layers - 1):
        fan_in = self.layer_sizes[i]
        fan_out = self.layer_sizes[i + 1]
        self.weights.append(rng.normal(mean, std, size=(fan_in, fan_out)))
        self.biases.append(np.zeros((1, fan_out)))
        self.grad_weights.append(np.zeros((fan_in, fan_out)))
        self.grad_biases.append(np.zeros((1, fan_out)))

```

5. `forward()`

Digunakan untuk melakukan forward propagation, yaitu proses menghitung output jaringan dari input yang diberikan. Secara garis besar, berikut adalah tahapan yang dilakukan:

- Menghitung kombinasi linear antara input dengan bobot dan bias dengan
$$z = aW + b$$
- Menerapkan fungsi aktivasi
$$a = f(z)$$
- Hasil aktivasi kemudian digunakan sebagai input untuk layer berikutnya. Setelah seluruh layer diproses, method ini mengembalikan output akhir dari jaringan.

```

def forward(self, X: np.ndarray) -> np.ndarray:
    self.z_list: List[np.ndarray] = []
    self.a_list: List[np.ndarray] = []
    a = X
    self.a_list.append(a)
    for i in range(self.num_layers - 1):
        z = np.dot(a, self.weights[i]) + self.biases[i]

```

```

        self.z_list.append(z)

        activation_fn = get_activation(self.activations[i])
        a = activation_fn(z)
        self.a_list.append(a)

    return a

```

6. backward()

Digunakan untuk melakukan proses backpropagation, yaitu menghitung gradien dari loss terhadap parameter jaringan. Secara garis besar, berikut adalah tahapan yang dilakukan:

- Menghitung loss terhadap output jaringan berdasarkan jenis loss function yang digunakan
- Kemudian akan dihitung nilai delta, yaitu turunan loss terhadap nilai pre-activation
- Kemudian gradien bobot dihitung dengan mengalikan transpose dari aktivasi layer sebelumnya dengan delta.
- Gradien bias dihitung dengan menjumlahkan delta pada seluruh sampel dalam batch.
- Gradien kemudian dipropagasikan ke layer sebelumnya hingga mencapai layer pertama.

```

def backward(self, y_true, loss_type="mse", reg_type=None, lambda_=0.0):
    y_pred = self.a_list[-1]
    if loss_type == "mse":
        dL_da = MSE.backward(y_true, y_pred)
    elif loss_type == "binary_crossentropy":
        dL_da = BinaryCrossEntropy.backward(y_true, y_pred)
    elif loss_type == "categorical_crossentropy":
        dL_da = CategoricalCrossEntropy.backward(y_true, y_pred)
    else:
        raise ValueError(f"Unknown loss type: {loss_type}")
    reg_grads = compute_regularization_gradient(self.weights, reg_type,
        lambda_)

    for i in reversed(range(self.num_layers - 1)):
        z = self.z_list[i]
        a_prev = self.a_list[i]
        activation_name = self.activations[i]

        if activation_name == "softmax":
            delta = softmax_derivative_vectorized(z, dL_da)
        else:
            activation_deriv = get_activation_derivative(activation_name)
            delta = dL_da * activation_deriv(z)

        self.grad_weights[i] = a_prev.T @ delta + reg_grads[i]
        self.grad_biases[i] = np.sum(delta, axis=0, keepdims=True)

```

```

        if i > 0:
            dL_da = delta @ self.weights[i].T

    return self.grad_weights, self.grad_biases

```

7. compute_loss()

Digunakan untuk menghitung nilai loss dari model. Loss terdiri atas dua komponen, yaitu data loss, yang merupakan loss utama yang dihitung berdasarkan perbedaan antara nilai prediksi dan nilai target dan regularization loss, yang digunakan untuk mengurangi kemungkinan overfitting dengan menambahkan penalti terhadap nilai bobot yang terlalu besar.

```

def compute_loss(self, y_true, y_pred, loss_type="mse", reg_type=None,
lambda_=0.0):
    if loss_type == "mse":
        data_loss = MSE.forward(y_true, y_pred)
    elif loss_type == "binary_crossentropy":
        data_loss = BinaryCrossEntropy.forward(y_true, y_pred)
    elif loss_type == "categorical_crossentropy":
        data_loss = CategoricalCrossEntropy.forward(y_true, y_pred)
    else:
        raise ValueError(f"Unknown loss type: {loss_type}")
    reg_loss = compute_regularization_loss(self.weights, reg_type, lambda_)
    return data_loss + reg_loss

```

8. _sgd_step()

Digunakan untuk memperbarui bobot dan bias menggunakan algoritma Stochastic Gradient Descent (SGD)

```

def _sgd_step(self, learning_rate: float) -> None:
    for i in range(self.num_layers - 1):
        self.weights[i] -= learning_rate * self.grad_weights[i]
        self.biases[i] -= learning_rate * self.grad_biases[i]

```

9. fit()

Merupakan method utama yang digunakan untuk melatih model neural network. Proses training dilakukan melalui beberapa langkah utama yaitu:

- Mengubah input menjadi array
- Melakukan validasi ukuran dataset
- Mengacak urutan data pada setiap epoch
- Membagi data menjadi beberapa mini batch. Pada setiap mini batch, dilakukan tiga tahap utama yaitu forward propagation, backward propagation, dan update parameter menggunakan SGD

- Setelah seluruh batch diproses dalam satu epoch, model akan menghitung nilai training loss dan validation loss.

```
def fit(
    self,
    X,
    y,
    X_val,
    y_val,
    batch_size,
    learning_rate,
    epochs,
    verbose,
    loss_type="mse",
    reg_type=None,
    lambda_=0.0,
    optimizer="sgd",
    beta1=0.9,
    beta2=0.999,
    adam_eps=1e-8,
):
    X = np.asarray(X)
    y = np.asarray(y)
    X_val = None if X_val is None else np.asarray(X_val)
    y_val = None if y_val is None else np.asarray(y_val)

    if X.shape[0] != y.shape[0]:
        raise ValueError("Jumlah sample X dan y harus sama.")
    if X_val is not None and y_val is not None and X_val.shape[0] != y_val.shape[0]:
        raise ValueError("Jumlah sample X_val dan y_val harus sama.")
    if batch_size <= 0:
        raise ValueError("batch_size harus > 0.")
    if epochs <= 0:
        raise ValueError("epochs harus > 0.")
    if verbose not in (0, 1):
        raise ValueError("verbose hanya boleh 0 atau 1.")
    if optimizer not in ("sgd", "adam"):
        raise ValueError("optimizer hanya boleh 'sgd' atau 'adam'.")
```

```

if not (0.0 < beta1 < 1.0 and 0.0 < beta2 < 1.0):
    raise ValueError("beta1 dan beta2 harus di rentang (0, 1).")
if adam_eps <= 0:
    raise ValueError("adam_eps harus > 0.")

num_samples = X.shape[0]
history = {"train_loss": [], "val_loss": []}
if optimizer == "adam":
    self._reset_adam_states()

for epoch in range(epochs):
    indices = np.random.permutation(num_samples)
    X_shuffled = X[indices]
    y_shuffled = y[indices]

    for start_idx in range(0, num_samples, batch_size):
        end_idx = min(start_idx + batch_size, num_samples)
        X_batch = X_shuffled[start_idx:end_idx]
        y_batch = y_shuffled[start_idx:end_idx]

        self.forward(X_batch)
        self.backward(
            y_true=y_batch,
            loss_type=loss_type,
            reg_type=reg_type,
            lambda_=lambda_,
        )
        if optimizer == "adam":
            self._adam_step(
                learning_rate=learning_rate,
                beta1=beta1,
                beta2=beta2,
                eps=adam_eps,
            )
        else:
            self._sgd_step(learning_rate)

    train_pred = self.forward(X)
    train_loss = self.compute_loss(

```

```

        y_true=y,
        y_pred=train_pred,
        loss_type=loss_type,
        reg_type=reg_type,
        lambda_=lambda_,
    )

    if X_val is not None and y_val is not None:
        val_pred = self.forward(X_val)
        val_loss = self.compute_loss(
            y_true=y_val,
            y_pred=val_pred,
            loss_type=loss_type,
            reg_type=reg_type,
            lambda_=lambda_,
        )
    else:
        val_loss = np.nan

    history["train_loss"].append(float(train_loss))
    history["val_loss"].append(float(val_loss))

    if verbose == 1:
        progress = (epoch + 1) / epochs
        bar_len = 30
        filled = int(bar_len * progress)
        bar = "#" * filled + "-" * (bar_len - filled)
        if np.isnan(val_loss):
            msg = (
                f"\rEpoch {epoch + 1}/{epochs} [{bar}] "
                f"train_loss={train_loss:.6f}"
            )
        else:
            msg = (
                f"\rEpoch {epoch + 1}/{epochs} [{bar}] "
                f"train_loss={train_loss:.6f} "
                f"val_loss={val_loss:.6f}"
            )
        print(msg, end="", flush=True)

```

```

if verbose == 1:
    print()

return history

```

10. `_reset_adam_state`

Inisialisasi ulang seluruh state Adam ke nol sebelum training baru, supaya history optimizer run sebelumnya tidak tercampur ke run berikutnya

```

def _reset_adam_states(self) -> None:
    self._adam_t = 0
    self._adam_m_w = [np.zeros_like(w) for w in self.weights]
    self._adam_v_w = [np.zeros_like(w) for w in self.weights]
    self._adam_m_b = [np.zeros_like(b) for b in self.biases]
    self._adam_v_b = [np.zeros_like(b) for b in self.biases]
    if self.use_rmsnorm:
        self._adam_m_g = [np.zeros_like(g) for g in self.rms_gamma]
        self._adam_v_g = [np.zeros_like(g) for g in self.rms_gamma]
    else:
        self._adam_m_g = []
        self._adam_v_g = []

```

11. `_adam_state`

Melakukan satu langkah update Adam untuk bobot, bias, dan gamma RMSNorm.

```

def _adam_step(
    self,
    learning_rate: float,
    beta1: float = 0.9,
    beta2: float = 0.999,
    eps: float = 1e-8,
) -> None:
    self._adam_t += 1
    t = self._adam_t

    for i in range(self.num_layers - 1):
        g_w = self.grad_weights[i]
        g_b = self.grad_biases[i]

        self._adam_m_w[i] = beta1 * self._adam_m_w[i] + (1.0 - beta1) *
g_w

```

```

        self._adam_v_w[i] = beta2 * self._adam_v_w[i] + (1.0 - beta2) *
(g_w ** 2)
        self._adam_m_b[i] = beta1 * self._adam_m_b[i] + (1.0 - beta1) *
g_b
        self._adam_v_b[i] = beta2 * self._adam_v_b[i] + (1.0 - beta2) *
(g_b ** 2)

        m_hat_w = self._adam_m_w[i] / (1.0 - beta1 ** t)
        v_hat_w = self._adam_v_w[i] / (1.0 - beta2 ** t)
        m_hat_b = self._adam_m_b[i] / (1.0 - beta1 ** t)
        v_hat_b = self._adam_v_b[i] / (1.0 - beta2 ** t)

        self.weights[i] -= learning_rate * m_hat_w / (np.sqrt(v_hat_w) +
eps)
        self.biases[i] -= learning_rate * m_hat_b / (np.sqrt(v_hat_b) +
eps)

        if self.use_rmsnorm:
            g_g = self.grad_rms_gamma[i]
            self._adam_m_g[i] = beta1 * self._adam_m_g[i] + (1.0 - beta1)
* g_g
            self._adam_v_g[i] = beta2 * self._adam_v_g[i] + (1.0 - beta2)
* (g_g ** 2)
            m_hat_g = self._adam_m_g[i] / (1.0 - beta1 ** t)
            v_hat_g = self._adam_v_g[i] / (1.0 - beta2 ** t)
            self.rms_gamma[i] -= learning_rate * m_hat_g /
(np.sqrt(v_hat_g) + eps)

```

1.2 Activation

Berikut merupakan implementasi fungsi aktivasi yang terdiri dari fungsi linear, ReLU, Sigmoid, Tanh, dan softmax. Masing-masing fungsi aktivasi memiliki implementasi turunan sehingga dapat menghitung *chain rule*-nya.

1.2.1. Linear

Berikut merupakan implementasi fungsi aktivasi linear:

```

def linear(z):
    return z

def linear_derivative(z):
    return np.ones_like(z)

```

1.2.2. ReLU

Berikut merupakan implementasi fungsi aktivasi ReLU:

```
def relu(z):  
    return np.maximum(0, z)  
  
def relu_derivative(z):  
    return (z > 0).astype(z.dtype)
```

1.2.3. Sigmoid

Berikut merupakan implementasi fungsi aktivasi sigmoid:

```
def sigmoid(z):  
    result = np.zeros_like(z, dtype=np.float64)  
    pos = z >= 0  
    neg = ~pos  
    result[pos] = 1.0 / (1.0 + np.exp(-z[pos]))  
    exp_z = np.exp(z[neg])  
    result[neg] = exp_z / (1.0 + exp_z)  
    return result  
  
def sigmoid_derivative(z):  
    s = sigmoid(z)  
    return s * (1 - s)
```

1.2.4. Tanh

Berikut merupakan implementasi fungsi aktivasi tanh:

```
def tanh(z):
    return np.tanh(z)

def tanh_derivative(z):
    return 1 - np.tanh(z) ** 2
```

1.2.5. Softmax

Berikut merupakan implementasi fungsi aktivasi softmax:

```
def softmax(z):
    if z.ndim == 1:
        exp_z = np.exp(z - np.max(z))
        return exp_z / np.sum(exp_z)
    else:
        exp_z = np.exp(z - np.max(z, axis=-1, keepdims=True))
        return exp_z / np.sum(exp_z, axis=-1, keepdims=True)

def softmax_derivative(z):
    s = softmax(z)
    if z.ndim == 1:
        return np.diag(s) - np.outer(s, s)
    else:
        batch_size, num_classes = s.shape
        identity = np.eye(num_classes)
        return s[:, :, None] * (identity[None, :, :] - s[:, None, :])

def softmax_derivative_vectorized(z, upstream_gradient):
    s = softmax(z)
    if z.ndim == 1:
        return s * (upstream_gradient - np.sum(s * upstream_gradient))
    else:
        sum_term = np.sum(s * upstream_gradient, axis=-1, keepdims=True)
        return s * (upstream_gradient - sum_term)
```

1.2.6. LeakyReLU

Berikut merupakan implementasi fungsi aktivasi LeakyReLU:

```
def leaky_relu(z, alpha=0.01):
    return np.where(z > 0, z, alpha * z)

def leaky_relu_derivative(z, alpha=0.01):
```

```
return np.where(z > 0, 1.0, alpha).astype(z.dtype)
```

1.2.7. Swish

Berikut merupakan implementasi fungsi aktivasi Swish:

```
def swish(z):  
    return z * sigmoid(z)  
  
def swish_derivative(z):  
    s = sigmoid(z)  
    return s + z * s * (1.0 - s)
```

1.2.6. Fungsi bantuan

Berikut adalah dictionary untuk mapping aktivasi ke fungsi dan fungsi get_activation sebagai entry point:

```
ACTIVATIONS = {  
    "linear": linear,  
    "relu": relu,  
    "leaky_relu": leaky_relu,  
    "sigmoid": sigmoid,  
    "tanh": tanh,  
    "swish": swish,  
    "softmax": softmax,  
}  
  
ACTIVATION_DERIVATIVES = {  
    "linear": linear_derivative,  
    "relu": relu_derivative,  
    "leaky_relu": leaky_relu_derivative,  
    "sigmoid": sigmoid_derivative,  
    "tanh": tanh_derivative,  
    "swish": swish_derivative,  
    "softmax": softmax_derivative,  
}  
  
def get_activation(name):  
    name = name.lower()  
    if name not in ACTIVATIONS:  
        raise ValueError(  
            f"Unknown activation: '{name}'. Available:  
{list(ACTIVATIONS.keys())}"  
        )  
    return ACTIVATIONS[name]
```

```
def get_activation_derivative(name):
    name = name.lower()
    if name not in ACTIVATION_DERIVATIVES:
        raise ValueError(
            f"Unknown activation: '{name}'. Available: {list(ACTIVATION_DERIVATIVES.keys())}"
        )
    return ACTIVATION_DERIVATIVES[name]
```

1.3 Losses

Bagian ini berisi kumpulan rumus untuk menghitung seberapa besar kesalahan prediksi model. Nilai kesalahan ini dipakai sebagai acuan saat training, dimana semakin kecil loss, semakin baik prediksi model terhadap data target. Selain menghitung nilai loss (forward), setiap class juga menyediakan turunan gradiennya (backward), sehingga bobot jaringan bisa diperbarui lewat proses backpropagation.

1.3.1. MSE

```
class MSE:
    @staticmethod
    def forward(y_true: np.ndarray, y_pred: np.ndarray) -> float:
        return np.mean((y_pred - y_true) ** 2)

    @staticmethod
    def backward(y_true: np.ndarray, y_pred: np.ndarray) -> np.ndarray:
        return 2 * (y_pred - y_true) / y_true.size
```

Class ini menghitung rata-rata kuadrat selisih antara nilai prediksi dan nilai sebenarnya. Cocok untuk kasus regresi karena sensitif pada error besar lebih kuat. Pada bagian backward, class ini mengembalikan gradien sederhana yang menunjukkan arah perbaikan prediksi agar lebih dekat ke target.

1.3.2. BinaryCrossEntropy

```
class BinaryCrossEntropy:
    @staticmethod
    def forward(y_true: np.ndarray, y_pred: np.ndarray) -> float:
        y_pred = np.clip(y_pred, 1e-15, 1 - 1e-15)
        return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

    @staticmethod
```

```
def backward(y_true: np.ndarray, y_pred: np.ndarray) -> np.ndarray:
    y_pred = np.clip(y_pred, 1e-15, 1 - 1e-15)
    return ((1 - y_true) / (1 - y_pred) - y_true / y_pred) / y_true.size
```

Class ini dipakai untuk klasifikasi biner, saat output model berupa probabilitas dua kelas. Di forward, prediksi lebih dulu diamankan dengan clipping agar tidak menghasilkan $\log(0)$, lalu dihitung loss cross-entropy biner. Di backward, class menghasilkan gradien terhadap probabilitas prediksi, sehingga model tahu bagaimana menyesuaikan output agar probabilitas kelas benar meningkat.

1.3.3. CategoricalCrossEntropy

```
class CategoricalCrossEntropy:
    @staticmethod
    def forward(y_true: np.ndarray, y_pred: np.ndarray) -> float:
        y_pred = np.clip(y_pred, 1e-15, 1 - 1e-15)
        return -np.sum(y_true * np.log(y_pred)) / y_true.shape[0]

    @staticmethod
    def backward(y_true: np.ndarray, y_pred: np.ndarray) -> np.ndarray:
        y_pred = np.clip(y_pred, 1e-15, 1 - 1e-15)
        return - (y_true / y_pred) / y_true.shape[0]
```

Class ini ditujukan untuk klasifikasi multi-class. Forward menghitung total cross-entropy per sampel lalu dirata-ratakan untuk satu batch. Sama seperti BinaryCrossEntropy, clipping dipakai untuk stabilitas numerik. Backward mengembalikan gradien untuk tiap kelas pada tiap sampel, sehingga model dapat memperbaiki distribusi probabilitas antar kelas.

1.4 Regularizers

1.4.1. Regularisasi L1

Berikut merupakan implementasi regulasi L1 untuk gradient dan loss

```
def l1_gradient(weights, lambda_):
    return lambda_ * np.sign(weights)

def l1_loss(weights, lambda_):
    return lambda_ * np.sum(np.abs(weights))
```

1.4.2. Regularisasi L2

Berikut merupakan implementasi regulasi L2 untuk gradient dan loss

```
def l2_gradient(weights, lambda_):
    return lambda_ * weights

def l2_loss(weights, lambda_):
```

```
return 0.5 * lambda_ * np.sum(weights**2)
```

1.4.3. Fungsi bantuan

Berikut adalah dictionary untuk mapping regulasi ke fungsi dan fungsi tambahan sebagai entry point:

```
REGULARIZERS = {
    "l1": {"gradient": l1_gradient, "loss": l1_loss},
    "l2": {"gradient": l2_gradient, "loss": l2_loss},
    None: {"gradient": lambda w, lam: 0, "loss": lambda w, lam: 0},
}

def get_regularizer(name):
    if name is None:
        return REGULARIZERS[None]
    name = name.lower()
    if name not in REGULARIZERS:
        raise ValueError(
            f"Unknown regularizer: '{name}'. Available: ['l1', 'l2', None]"
        )
    return REGULARIZERS[name]

def compute_regularization_gradient(weights_list, reg_type, lambda_):
    if reg_type is None or lambda_ == 0:
        return [np.zeros_like(w) for w in weights_list]

    reg = get_regularizer(reg_type)
    return [reg["gradient"](w, lambda_) for w in weights_list]

def compute_regularization_loss(weights_list, reg_type, lambda_):
    if reg_type is None or lambda_ == 0:
        return 0.0

    reg = get_regularizer(reg_type)
    return sum(reg["loss"](w, lambda_) for w in weights_list)
```

1.5 Data Preparation

Bagian ini berperan sebagai tahap persiapan data sebelum model FFNN dilatih. Intinya, file ini membaca data mentah dari CSV, mengubah data kategorikal menjadi angka, menormalkan fitur agar skalanya seimbang, lalu membagi data menjadi data latih dan data validasi. Dengan alur ini, data yang masuk ke model jadi lebih rapi, konsisten, dan siap diproses saat training.

```
def label_encode(column):
```

```

unique_vals = sorted(list(set(column)))
val_to_int = {val: i for i, val in enumerate(unique_vals)}
return np.array([val_to_int[val] for val in column]), val_to_int

```

Fungsi `label_encode` mengubah nilai kategori menjadi angka integer secara konsisten. Selain hasil encoding, fungsi ini juga mengembalikan mapping kategori ke angka, sehingga konversi yang dipakai bisa diketahui dengan jelas.

```

def load_and_preprocess_data(filepath, test_size=0.2, random_state=42):
    with open(filepath, 'r', encoding='utf-8') as f:
        reader = csv.reader(f)
        header = next(reader)
        data = list(reader)

    data = np.array(data)

    categorical_cols = [2, 3, 4, 8, 9]
    numerical_cols = [0, 1, 5, 6, 7, 10]
    target_col = 11

    # Encode categorical
    encoded_features = []
    for col_idx in categorical_cols:
        encoded, mapping = label_encode(data[:, col_idx])
        encoded_features.append(encoded)

    encoded_features = np.column_stack(encoded_features)

    # Parse numerical
    numerical_features = data[:, numerical_cols].astype(float)

    # Combine features
    X = np.hstack((numerical_features, encoded_features))

    # Encode Target
    y, y_mapping = label_encode(data[:, target_col])

    # Normalize features (Standard Scaler)
    mean = np.mean(X, axis=0)
    std = np.std(X, axis=0)
    std[std == 0] = 1
    X = (X - mean) / std

```

```

# Train/Val split
np.random.seed(random_state)
indices = np.arange(X.shape[0])
np.random.shuffle(indices)

split_idx = int(X.shape[0] * (1 - test_size))
train_idx, val_idx = indices[:split_idx], indices[split_idx:]

X_train, X_val = X[train_idx], X[val_idx]
y_train, y_val = y[train_idx], y[val_idx]

return X_train, y_train, X_val, y_val

```

Fungsi `load_and_preprocess_data` adalah pipeline utama preprocessing. Langkahnya dimulai dari membaca CSV, memisahkan kolom kategorikal dan numerik, melakukan encoding pada fitur kategorikal, menggabungkan semua fitur, meng-encode label target, melakukan normalisasi standar (mean 0, std 1), lalu melakukan pembagian train/validation dengan random seed agar hasil split reproducible.

1.6 RMS Normalization

Bagian ini mencakup proses forward pass (normalisasi input) dan backward pass (perhitungan gradien untuk backpropagation). Kedua fungsi ini menjadi komponen inti yang dipasang di dalam arsitektur FFNN agar model dapat menggunakan RMSNorm selama pelatihan.

```

def rmsnorm_forward(x: np.ndarray, gamma: np.ndarray, eps: float = 1e-8) ->
np.ndarray:
    x_sq_mean = np.mean(x**2, axis=-1, keepdims=True)
    rms = np.sqrt(x_sq_mean + eps)
    x_norm = x / rms
    return x_norm * gamma

```

Fungsi ini bertugas menormalisasi vektor input `x` menggunakan akar rata-rata kuadrat (Root Mean Square). Fungsi ini menghitung rata-rata kuadrat dari setiap elemen input, mengambil akar kuadratnya dengan tambahan nilai `eps` yang sangat kecil untuk menghindari pembagian dengan nol, lalu membagi input dengan nilai RMS tersebut. Hasil normalisasi kemudian dikalikan dengan parameter `gamma`, parameter yang dapat dipelajari selama training, agar model tetap bisa menyesuaikan skala output secara adaptif.

```

def rmsnorm_backward(
    dout: np.ndarray, x: np.ndarray, gamma: np.ndarray, eps: float = 1e-8
) -> tuple[np.ndarray, np.ndarray]:

```

```

x_sq_mean = np.mean(x**2, axis=-1, keepdims=True)
rms = np.sqrt(x_sq_mean + eps)
x_norm = x / rms

# Gradient w.r.t gamma
dgamma = np.sum(dout * x_norm, axis=0, keepdims=True)

# Gradient w.r.t x
dy = dout * gamma
mean_dy_x = np.mean(dy * x, axis=-1, keepdims=True)
dx = (dy / rms) - (x * mean_dy_x) / (rms**3)

return dx, dgamma

```

Fungsi ini menghitung dua gradien yang dibutuhkan saat backpropagation, yaitu gradien terhadap gamma (dgamma) dan gradien terhadap input x (dx), berdasarkan gradien yang datang dari layer berikutnya (dout). Gradien dgamma diperoleh dengan mengalikan dout dengan input yang sudah ternormalisasi, lalu menjumlahkannya ke seluruh sampel dalam batch. Sementara itu, dx sedikit lebih kompleks karena normalisasi membuat setiap elemen input saling bergantung satu sama lain, perhitungannya terdiri dari dua suku, yaitu suku yang meneruskan gradien secara langsung melalui pembagian RMS, dan suku koreksi yang memperhitungkan efek bahwa perubahan pada satu elemen x turut memengaruhi nilai RMS secara keseluruhan.

1.7 Automatic Differentiation

1.7.1 Implementasi Autodiff

```

class Value:
    def __init__(self, data, _children=(), _op=""):
        self.data = float(data)
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)
        self._op = _op

    def __repr__(self):
        return f"Value(data={self.data}, grad={self.grad})"

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), "+")

        def _backward():
            self.grad += 1.0 * out.grad
            other.grad += 1.0 * out.grad

        out._backward = _backward
        return out

```

```

def __radd__(self, other):
    return self + other

def __mul__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data * other.data, (self, other), "*")

    def _backward():
        self.grad += other.data * out.grad
        other.grad += self.data * out.grad

    out._backward = _backward
    return out

def __rmul__(self, other):
    return self * other

def __neg__(self):
    out = Value(-self.data, (self,), "neg")

    def _backward():
        self.grad += -1.0 * out.grad

    out._backward = _backward
    return out

def __sub__(self, other):
    return self + (-other)

def __rsub__(self, other):
    return other + (-self)

def __truediv__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    return self * other**-1

def __rtruediv__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    return other * self**-1

def __pow__(self, power):
    assert isinstance(power, (int, float))
    out = Value(self.data**power, (self,), f"**{power}")

    def _backward():
        self.grad += power * (self.data ** (power - 1)) * out.grad

    out._backward = _backward
    return out

def exp(self):
    out = Value(float(np.exp(self.data)), (self,), "exp")

    def _backward():
        self.grad += out.data * out.grad

    out._backward = _backward
    return out

```

```

def log(self):
    out = Value(float(np.log(self.data)), (self,), "log")

    def _backward():
        self.grad += (1.0 / self.data) * out.grad

    out._backward = _backward
    return out

def tanh(self):
    t = float(np.tanh(self.data))
    out = Value(t, (self,), "tanh")

    def _backward():
        self.grad += (1 - t**2) * out.grad

    out._backward = _backward
    return out

def relu(self):
    out = Value(self.data if self.data > 0 else 0.0, (self,), "relu")

    def _backward():
        self.grad += (1.0 if self.data > 0 else 0.0) * out.grad

    out._backward = _backward
    return out

def sigmoid(self):
    s = 1.0 / (1.0 + float(np.exp(-self.data)))
    out = Value(s, (self,), "sigmoid")

    def _backward():
        self.grad += s * (1 - s) * out.grad

    out._backward = _backward
    return out

def backward(self):
    topo = []
    visited = set()

    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._prev:
                build_topo(child)
            topo.append(v)

    build_topo(self)
    self.grad = 1.0
    for v in reversed(topo):
        v._backward()

```

Kelas Value adalah inti dari implementasi autodiff. Setiap objek Value menyimpan sebuah nilai skalar beserta gradiennya, dan setiap operasi matematis seperti penjumlahan, perkalian, perpangkatan, hingga fungsi aktivasi seperti relu, sigmoid, dan tanh, secara otomatis mendefinisikan fungsi `_backward`-nya masing-masing. Fungsi inilah yang menghitung bagaimana gradien dari output diteruskan kembali ke

setiap input operasi tersebut. Ketika metode `backward()` dipanggil pada node output, kelas ini membangun urutan topologi dari seluruh graf komputasi, lalu menelusuri ulang setiap operasi dari belakang ke depan untuk mengakumulasi gradien di setiap node, proses inilah yang dikenal sebagai reverse-mode automatic differentiation.

1.7.2 Integrasi FFNN

Kelas `AutodiffFFNN` mengintegrasikan `Value` ke dalam class `FFNN` yang sebelumnya sudah dibuat. Arsitekturnya dirancang selaras dengan `FFNN` yang sebelumnya yaitu menerima konfigurasi layer dan fungsi aktivasi yang sama, mendukung aktivasi `relu`, `sigmoid`, `tanh`, dan `softmax`, serta menyediakan fungsi `loss mse_loss` dan `categorical_crossentropy_loss`. Proses forward pass-nya membangun graf komputasi secara otomatis dari setiap operasi perkalian bobot dan penjumlahan bias, sehingga begitu `backward()` dipanggil pada nilai loss, seluruh gradien terhadap bobot dan bias langsung tersedia tanpa perlu menurunkan rumus secara manual. Metode `set_params` dan `get_grads` memungkinkan bobot dari `FFNN` utama disalin ke `AutodiffFFNN`, sehingga kedua model dapat dibandingkan gradiennya pada kondisi yang identik.

1.8 Adam (Adaptive Moment Estimation)

Adam adalah algoritma optimisasi yang menggabungkan momentum dan adaptive learning rate dengan cara menghitung rata-rata gradien (first moment) dan rata-rata kuadrat gradien (second moment). Nilai tersebut kemudian dikoreksi bias dan digunakan untuk memperbarui parameter secara adaptif. Pada implementasi ini, Adam digunakan untuk memperbarui bobot, bias, serta parameter `RMSNorm` sehingga proses training menjadi lebih stabil dan konvergen lebih cepat dibandingkan metode SGD biasa.

Proses algoritma Adam dapat dilihat secara berikut, untuk setiap parameter θ (bias)

- Menghitung momentum (first moment)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

- Menghitung RMS (second moment)

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

- Bias correction

$$m_t = \frac{m_t}{1 - \beta_1^t}$$

$$v_t = \frac{v_t}{1 - \beta_2^t}$$

- Update parameter

$$\theta = \theta - \alpha \cdot \frac{m_t}{\sqrt{v_t} + \epsilon}$$

Pada implementasi `FFNN` ini, algoritma Adam diintegrasikan dalam method `_adam_step`, yang dipanggil selama proses training di dalam fungsi `fit()` ketika parameter optimizer diset menjadi "adam". Setiap kali `_adam_step` dijalankan, pertama-tama dilakukan peningkatan timestep yang disimpan pada variabel `self._adam_t`, yang merepresentasikan iterasi ke-t dalam proses optimisasi. Selanjutnya, gradien dari bobot dan bias pada setiap layer diambil dari `self.grad_weights[i]` dan `self.grad_biases[i]`, yang merupakan hasil dari proses backpropagation.

Gradien tersebut kemudian digunakan untuk menghitung momentum (first moment) dengan memperbarui nilai `self._adam_m_w[i]` dan `self._adam_m_b[i]`, menggunakan parameter `beta1` sebagai faktor peluruhan. Setelah itu, dihitung juga second moment (RMS) dengan memperbarui `self._adam_v_w[i]` dan `self._adam_v_b[i]`, menggunakan `beta2` dan kuadrat dari gradien. Kedua nilai ini kemudian dikoreksi menggunakan teknik bias correction, sehingga diperoleh estimasi yang tidak bias, yaitu `m_hat` dan `v_hat`.

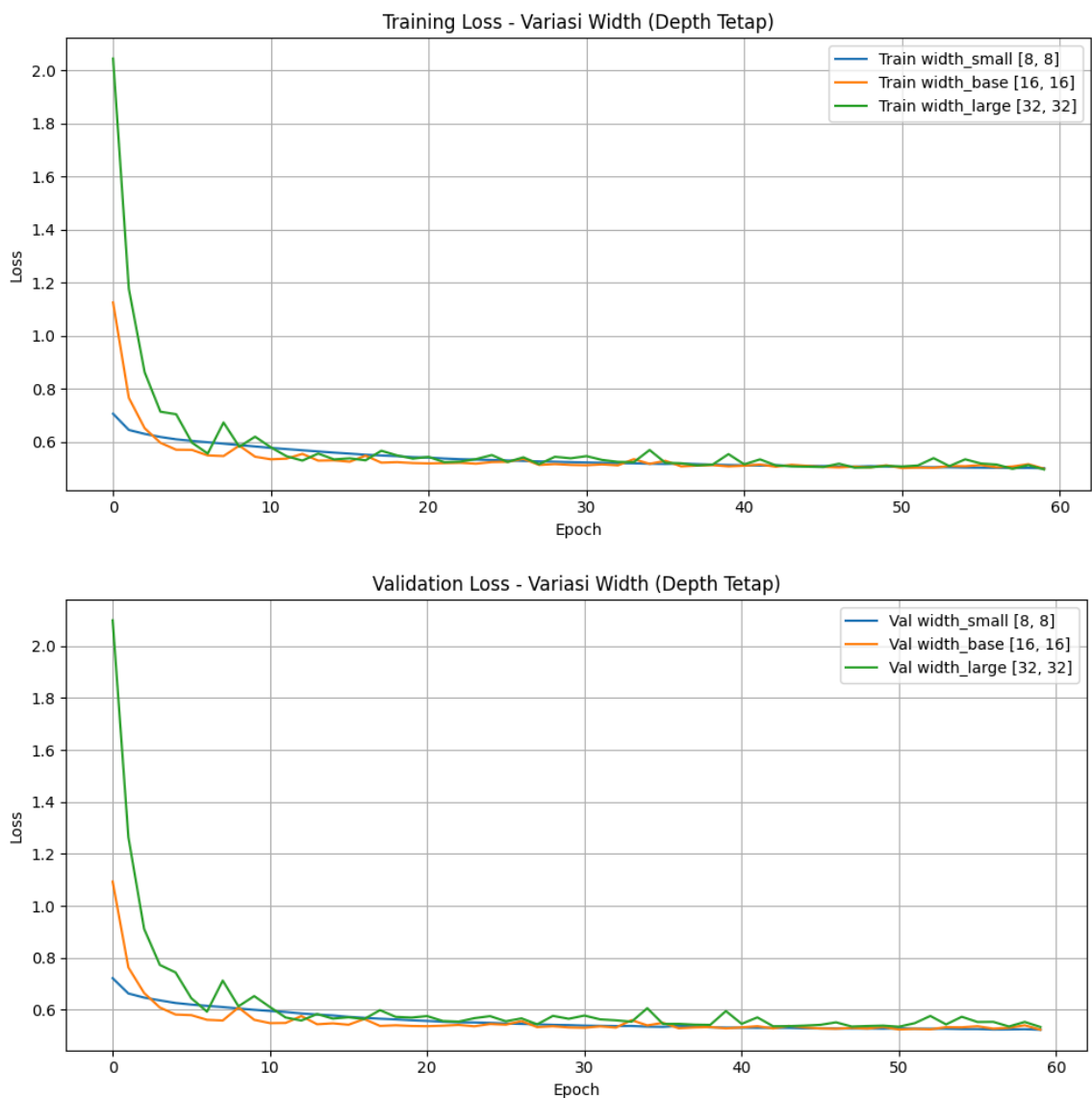
Setelah mendapatkan nilai tersebut, parameter model diperbarui menggunakan rumus Adam, yaitu bobot dan bias dikurangi dengan hasil pembagian antara `m_hat` dan akar dari `v_hat` yang ditambah epsilon untuk menjaga stabilitas numerik.

BAB 2

HASIL PENGUJIAN

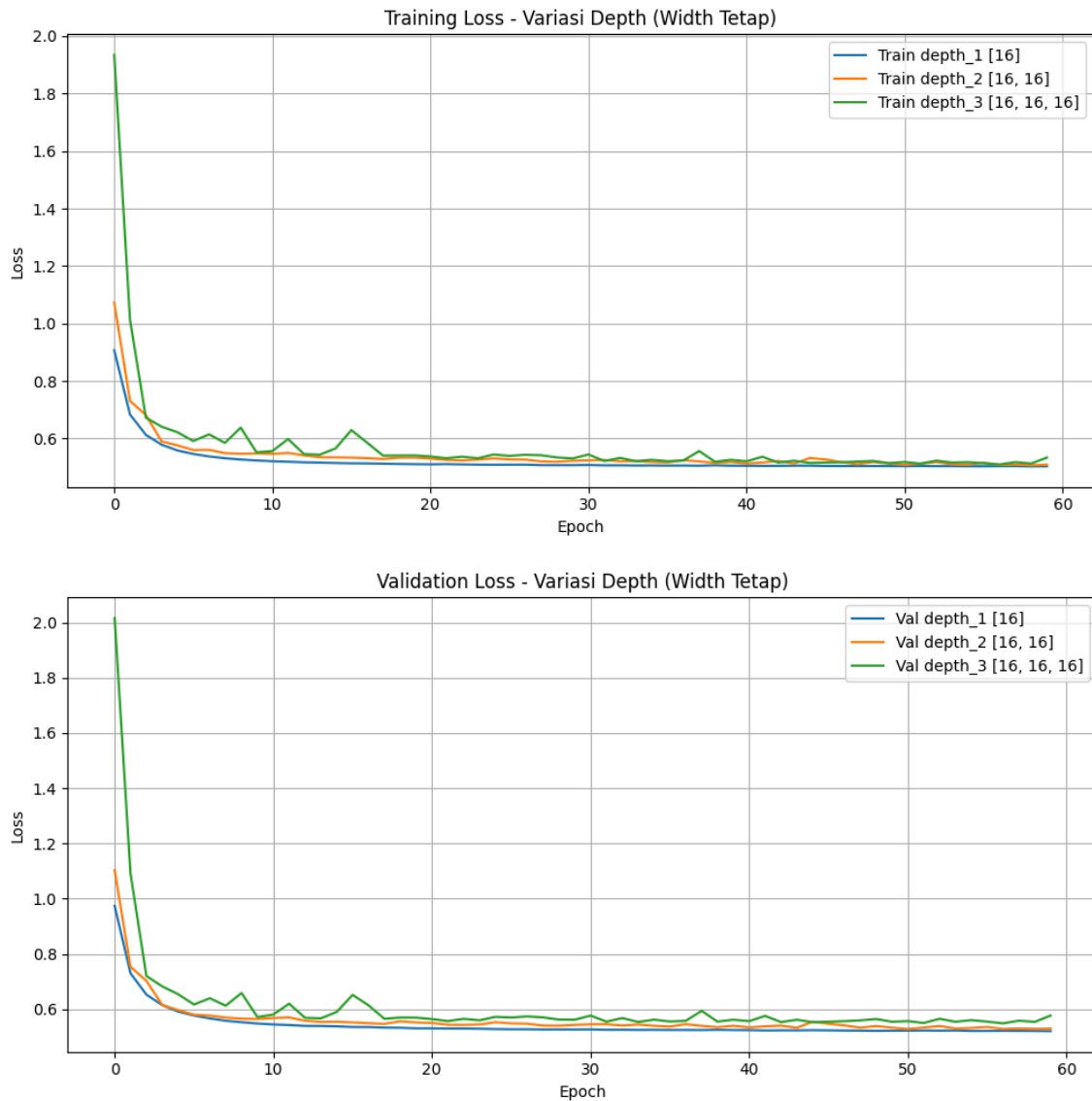
2.1 Pengaruh Depth dan Width

a. Depth Tetap



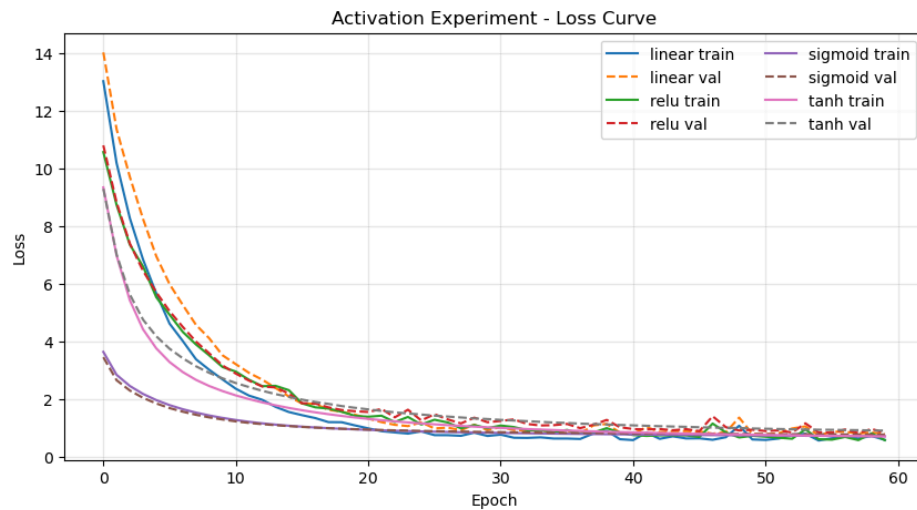
Width tidak memberikan pengaruh signifikan pada dataset ini. Model kecil [8, 8] maupun besar [32, 32] akhirnya belajar hal yang sama. Ini mengindikasikan bahwa bottleneck bukan di kapasitas model, melainkan kemungkinan besar di kualitas/kesulitan datanya sendiri, semua model "stuck" di loss yang sama karena data memang tidak memberikan sinyal yang lebih kuat dari itu.

b. Width Tetap



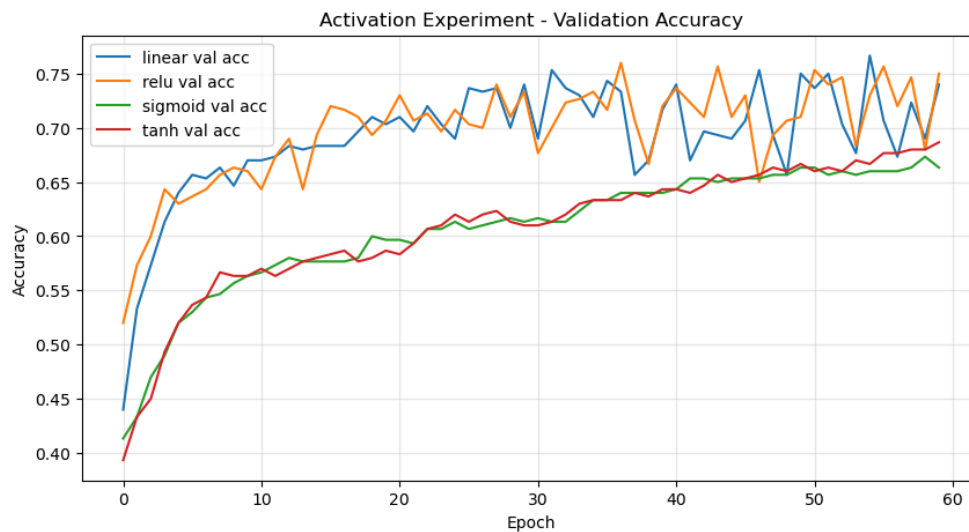
Berbeda dengan width, depth memberikan pengaruh yang lebih terlihat. Menambah layer justru memperburuk performa di dataset ini, model yang lebih dalam lebih sulit dioptimasi, lebih tidak stabil, dan berakhir dengan val loss lebih tinggi. Ini konsisten dengan fenomena degradasi pada jaringan dalam tanpa teknik khusus seperti batch normalization atau skip connection. Untuk dataset ini, depth_1 [16] adalah arsitektur paling efektif.

2.2 Pengaruh Fungsi Aktivasi



Pada grafik loss curve, seluruh fungsi aktivasi menunjukkan penurunan nilai loss yang cukup signifikan pada awal pelatihan. Model dengan fungsi aktivasi linear, tanh, dan ReLU mengalami penurunan loss yang lebih cepat dibandingkan fungsi lainnya, yang menunjukkan bahwa model dapat meminimalkan kesalahan prediksi dengan lebih efisien pada tahap awal pelatihan.

Model dengan fungsi aktivasi sigmoid juga menunjukkan penurunan loss secara bertahap, namun kurvanya cenderung lebih lambat dibandingkan yang lain. Hal ini dapat disebabkan oleh karakteristik fungsi aktivasi tersebut yang dapat mengalami vanishing gradient, sehingga proses pembelajaran berlangsung lebih lambat.



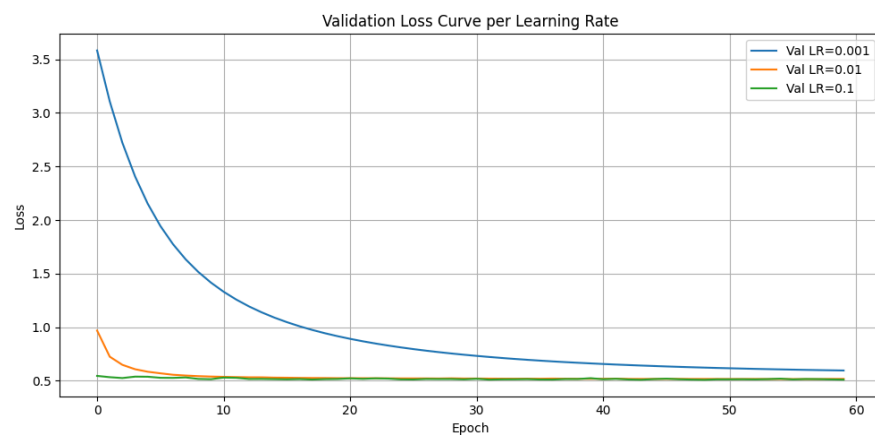
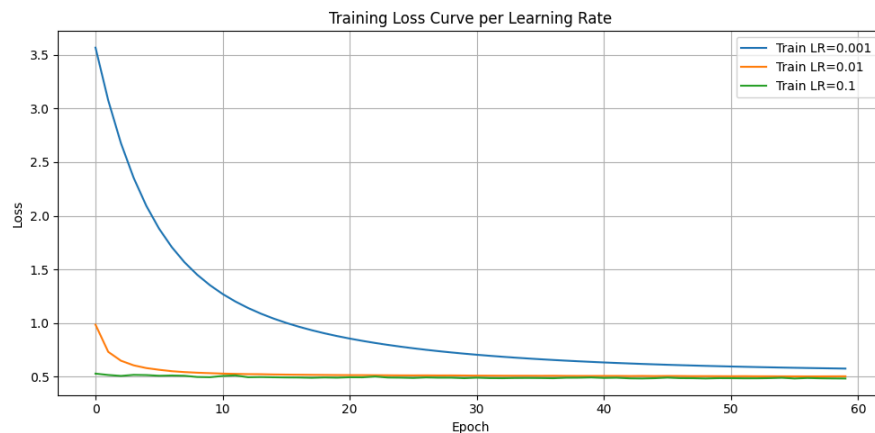
Pada grafik validation accuracy, model dengan fungsi aktivasi linear menunjukkan peningkatan akurasi yang cukup cepat pada awal pelatihan dan mencapai nilai sekitar 0.74–0.76 pada beberapa epoch. Meskipun demikian, kurva akurasi terlihat sedikit berfluktuasi pada epoch selanjutnya, yang menunjukkan bahwa performa model tidak sepenuhnya stabil selama proses pelatihan.

Model dengan fungsi aktivasi ReLU juga menunjukkan performa yang sangat baik dengan nilai akurasi validasi yang relatif tinggi dan stabil sepanjang pelatihan. Kurva akurasi ReLU sering berada di antara 0.72 hingga 0.75, yang menunjukkan bahwa fungsi aktivasi ini mampu membantu model mempelajari representasi data secara efektif.

Sementara itu, fungsi aktivasi sigmoid dan tanh menunjukkan peningkatan akurasi yang lebih lambat dibandingkan linear dan ReLU. Pada akhir pelatihan, akurasi kedua fungsi ini berada pada kisaran 0.66–0.68. Hal ini menunjukkan bahwa kedua fungsi tersebut masih mampu mempelajari pola data, namun performanya relatif lebih rendah dalam eksperimen ini.

2.3 Pengaruh Learning Rate

Eksperimen ini menguji tiga nilai learning rate yang berbeda, yaitu 0.001, 0.01, dan 0.1, menggunakan arsitektur FFNN yang identik, satu hidden layer dengan 16 neuron, aktivasi ReLU, serta sigmoid pada output layer. Ketiga model dilatih dengan kondisi yang sepenuhnya sama, yaitu 60 epoch, batch size 32, inisialisasi bobot normal, dan tanpa regularisasi. Dengan menyamakan semua variabel kecuali learning rate, perubahan performa yang muncul dapat langsung dikaitkan dengan perbedaan nilai learning rate tersebut. Perbandingan dilakukan melalui kurva training loss dan validation loss di setiap epoch, serta akurasi akhir pada data validasi.



```

FFNN final prediction metrics per learning rate
lr | final_train_loss | final_val_loss | final_val_accuracy
0.001 | 0.576909 | 0.596134 | 0.688500
0.01 | 0.505192 | 0.518005 | 0.733000
0.1 | 0.484465 | 0.510687 | 0.743500

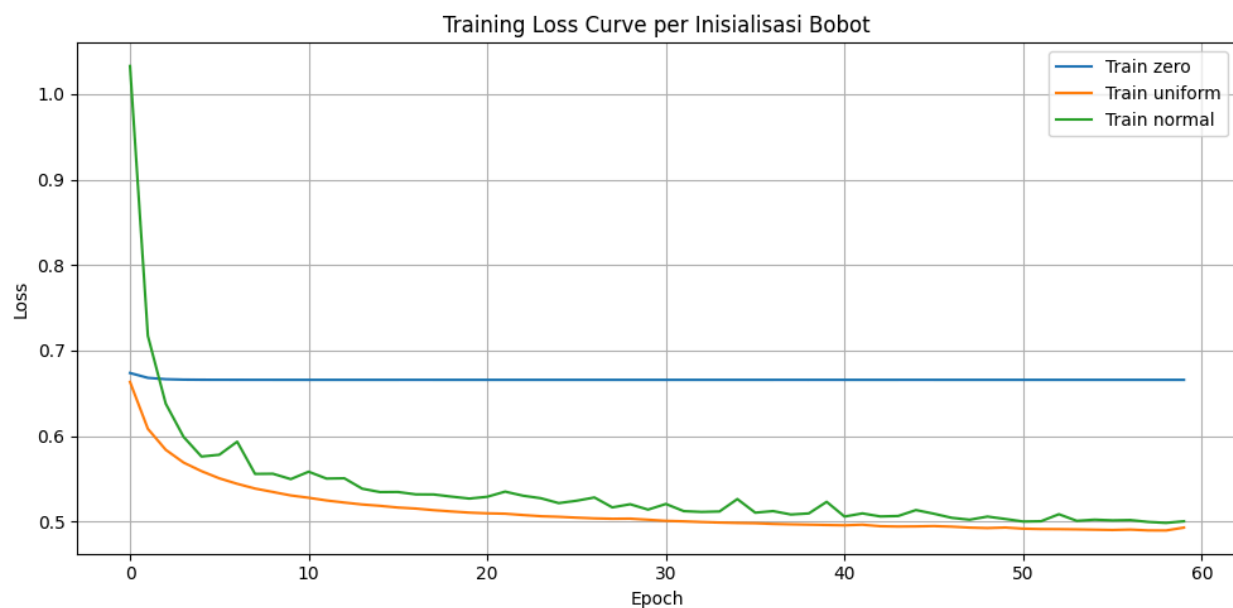
Best learning rate by final val accuracy: 0.1

```

Dari kurva loss dan tabel ringkasan, terlihat pola yang cukup jelas bahwa semakin besar learning rate, semakin cepat model belajar dan semakin rendah loss yang dicapai dalam jumlah epoch yang sama. Model dengan learning rate 0.001 menunjukkan penurunan loss yang paling lambat dan stagnan, training loss-nya hanya turun ke 0.5769 dan val accuracy-nya paling rendah di angka 68.85%. Ini biasa terjadi pada kasus learning rate yang terlalu kecil, di mana model butuh jauh lebih banyak epoch untuk konvergen.

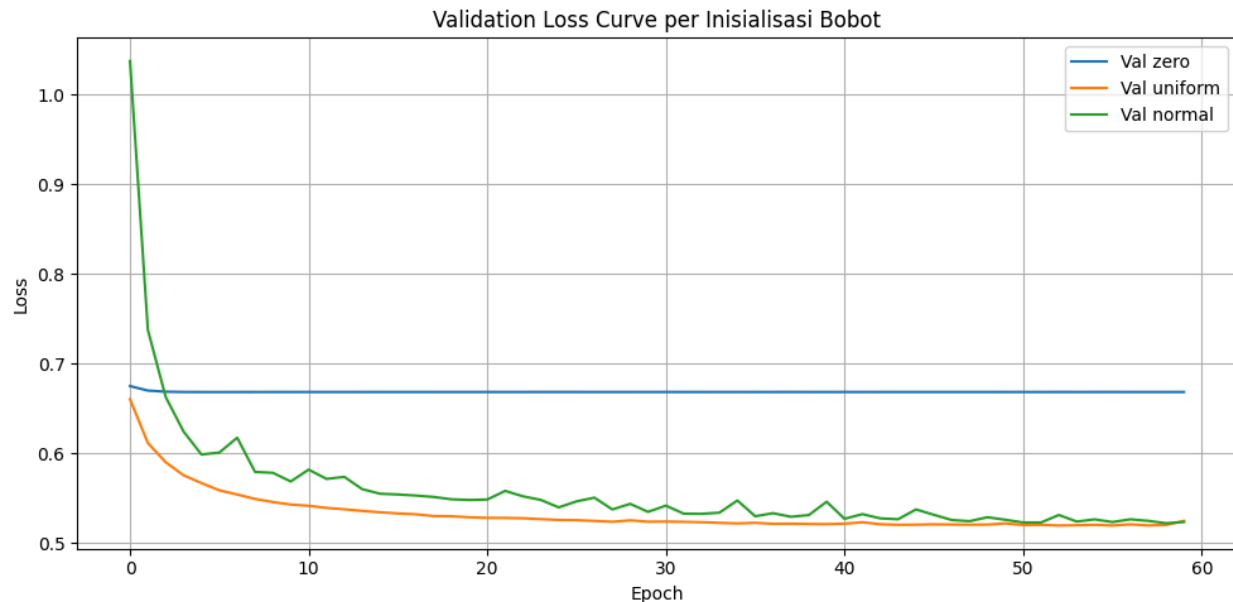
Sebaliknya, learning rate 0.1 menghasilkan penurunan loss yang paling agresif dan mencapai val accuracy tertinggi sebesar 74.35%, diikuti learning rate 0.01 dengan akurasi 73.30%. Meskipun learning rate 0.1 terbilang besar, kurva validasinya tidak menunjukkan tanda-tanda divergensi atau lonjakan loss yang signifikan, artinya model masih stabil dan tidak overshoot. Hal ini menunjukkan bahwa untuk dataset dan arsitektur yang digunakan, learning rate 0.1 merupakan pilihan terbaik di antara tiga nilai yang diuji.

2.4 Pengaruh Inisialisasi Bobot



Berdasarkan kurva training loss, terdapat perbedaan yang sangat mencolok antara ketiga metode inisialisasi. Inisialisasi zero menghasilkan kurva yang sepenuhnya datar pada nilai sekitar 0,67 sepanjang 60 epoch, yang menandakan bahwa model gagal melakukan proses pembelajaran sama sekali. Hal ini terjadi karena seluruh neuron menerima gradien yang identik sehingga simetri antar neuron tidak pernah terpecah, membuat bobot tidak pernah diperbarui secara bermakna.

Sementara itu, inisialisasi uniform menunjukkan penurunan loss yang paling mulus dan konsisten, berhasil mencapai nilai akhir terendah sekitar 0,49. Inisialisasi normal turun lebih cepat di awal namun disertai fluktuasi yang lebih tinggi, dan berkonvergensi pada nilai yang sedikit lebih tinggi dibanding uniform (~0,50).



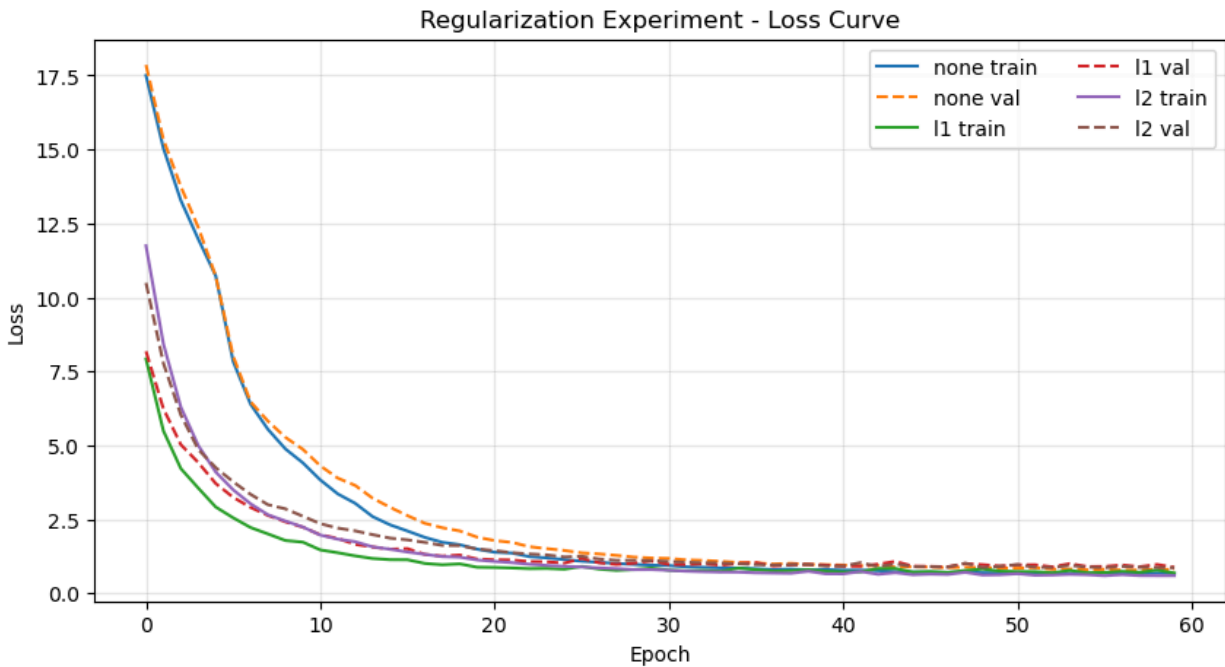
Pola pada validation loss sangat konsisten dengan training loss. Inisialisasi zero tetap stagnan di sekitar 0,67, mempertegas bahwa model tidak mampu menggeneralisasi apapun. Inisialisasi uniform kembali menghasilkan kurva paling stabil dengan nilai akhir terbaik (~0,523), sementara normal menghasilkan nilai yang sangat berdekatan (~0,524) namun dengan fluktuasi lebih tinggi sepanjang proses pelatihan. Tidak ditemukan tanda-tanda overfitting yang signifikan pada kedua inisialisasi yang berfungsi.

Inisialisasi bobot terbukti memberikan pengaruh yang sangat signifikan terhadap kemampuan model untuk belajar. Inisialisasi zero merupakan pendekatan yang harus dihindari karena menyebabkan model gagal berlatih secara menyeluruh. Di antara metode yang menghasilkan pembelajaran efektif, inisialisasi uniform menunjukkan stabilitas yang sedikit lebih baik dibandingkan normal, meskipun nilai konvergensi akhir keduanya hampir setara. Dengan demikian, pemilihan metode inisialisasi yang tepat merupakan faktor kritis dalam memastikan proses pelatihan jaringan saraf dapat berjalan dengan baik.

```
Ringkasan metrik akhir
init_method | final_train_loss | final_val_loss | final_val_accuracy
zero | 0.665812 | 0.668346 | 0.611000
uniform | 0.493047 | 0.524430 | 0.733000
normal | 0.500430 | 0.523265 | 0.739000

Best by validation accuracy: normal
Best by validation loss : normal
```

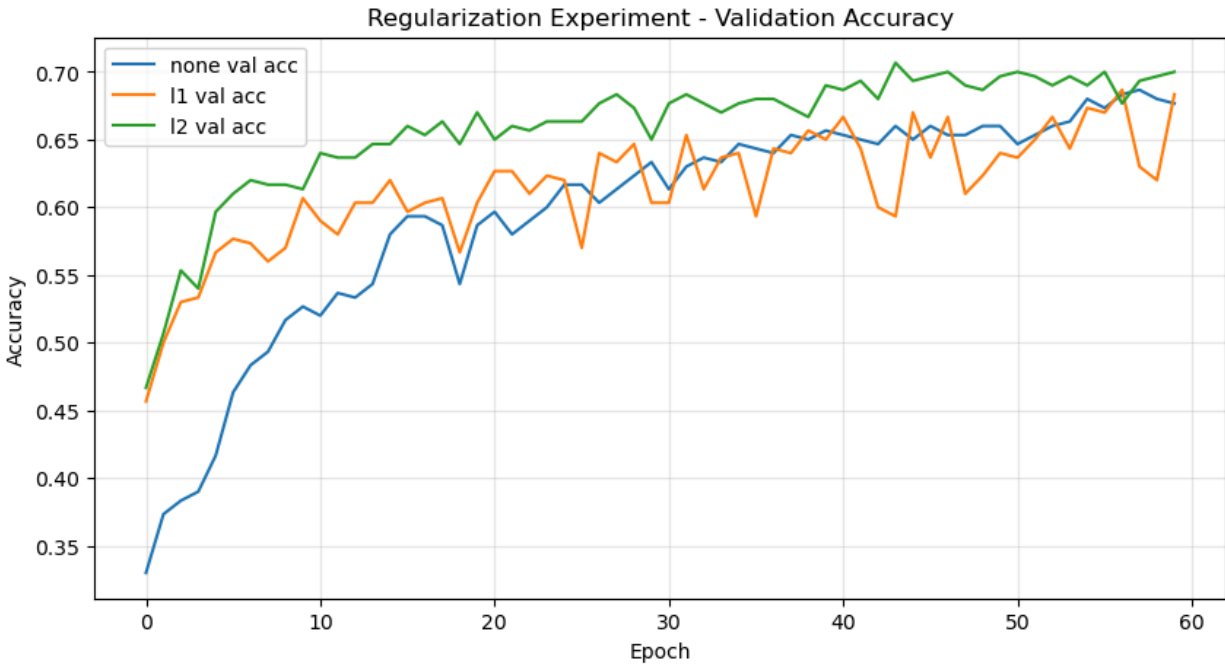
2.5 Pengaruh Regularisasi



Pada grafik validation loss, model tanpa regularisasi menunjukkan penurunan loss secara bertahap selama pelatihan, tapi kurvanya tidak stabil. Hal ini mengindikasikan bahwa meskipun kesalahan prediksi berkurang, model masih berpotensi mengalami overfitting karena tidak adanya penalti terhadap bobot yang terlalu besar.

Pada model dengan L1 regularization, kurva loss juga menunjukkan penurunan tetapi disertai beberapa fluktuasi. Fluktuasi ini sejalan dengan karakteristik L1 regularization yang membuat beberapa bobot menjadi nol, sehingga proses penyesuaian parameter selama pelatihan dapat berubah secara lebih tajam.

Sedangkan pada L2 regularization, grafik loss menunjukkan penurunan yang lebih konsisten dan stabil dibandingkan metode lainnya. Hal ini menunjukkan bahwa L2 regularization mampu mengontrol besarnya bobot model secara lebih efektif sehingga proses pelatihan menjadi lebih stabil dan menghasilkan kesalahan prediksi yang lebih kecil.



Pada grafik validation accuracy, model tanpa regularisasi menunjukkan peningkatan akurasi secara bertahap dari sekitar 0.33 pada awal pelatihan hingga mencapai kisaran 0.65–0.68 pada akhir epoch. Hal ini menunjukkan bahwa model mampu mempelajari pola dari data, meskipun peningkatan akurasinya tidak setinggi metode regularisasi.

Model dengan L1 regularization menunjukkan nilai akurasi validasi yang sedikit lebih tinggi dibandingkan tanpa regularisasi, dengan nilai akhir berada pada kisaran 0.66–0.68. Namun, kurva akurasi terlihat lebih berfluktuasi selama proses pelatihan karena L1 regularization mendorong beberapa bobot menjadi nol sehingga model menjadi lebih sederhana.

Sementara itu, model dengan L2 regularization menghasilkan performa terbaik. Kurva validation accuracy meningkat secara lebih stabil dan mencapai nilai sekitar 0.70 pada akhir pelatihan. Hal ini menunjukkan bahwa L2 regularization mampu membantu model melakukan generalisasi dengan lebih baik terhadap data validasi.

2.6 Pengaruh Normalisasi RMSNorm

Eksperimen ini membandingkan dua model FFNN yang identik secara arsitektur, tiga layer (input → 32 neuron → 16 neuron → output), aktivasi ReLU pada hidden layer dan Softmax pada output, serta inisialisasi bobot normal. Satu-satunya perbedaan di antara keduanya adalah penggunaan RMSNorm, dimana satu model dilatih tanpa normalisasi sama sekali, satu lagi menggunakan RMSNorm di setiap hidden layer. Kedua model dilatih dengan kondisi yang persis sama, learning rate 0.01, batch size 32, dan 50 epoch, pada dataset klasifikasi yang sama. Perbandingan dilakukan dari tiga sudut pandang, yaitu hasil prediksi akhir (akurasi), dinamika pelatihan melalui kurva loss, serta perilaku internal model melalui distribusi bobot dan gradien.

```
# Train model WITHOUT RMSNorm
model_base = FFNN(layers, activations, weight_initial='normal', use_rmsnorm=False)
history_base = model_base.fit(
    X_train, y_train_oh, X_val, y_val_oh,
    batch_size=batch_size,
    learning_rate=learning_rate,
    epochs=epochs,
    verbose=1,
    loss_type='categorical_crossentropy'
)
✓ 1.0s Python

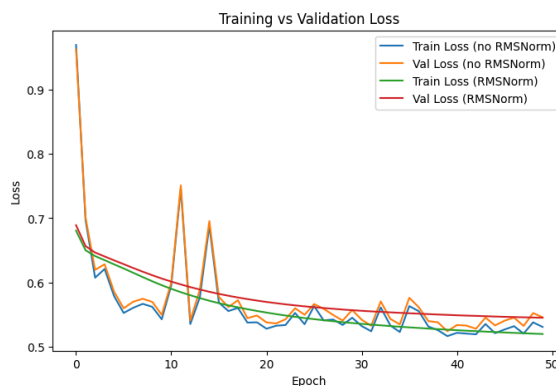
Epoch 50/50 [#####] train_loss=0.531170 val_loss=0.546222

# Train model WITH RMSNorm
model_rms = FFNN(layers, activations, weight_initial='normal', use_rmsnorm=True)
history_rms = model_rms.fit(
    X_train, y_train_oh, X_val, y_val_oh,
    batch_size=batch_size,
    learning_rate=learning_rate,
    epochs=epochs,
    verbose=1,
    loss_type='categorical_crossentropy'
)
✓ 2.1s Python

Epoch 50/50 [#####] train_loss=0.520235 val_loss=0.545476
```

Accuracy (no RMSNorm): 0.7160
Accuracy (RMSNorm): 0.7230

Model dengan RMSNorm menghasilkan akurasi validasi sebesar 72.30%, sedikit lebih tinggi dibandingkan model tanpa RMSNorm yang mencapai 71.60%. Meskipun selisihnya hanya 0.7%, peningkatan ini konsisten dengan penurunan loss yang lebih baik, training loss akhir model RMSNorm (0.5202) lebih rendah dibanding model tanpa normalisasi (0.5312), begitu pula validation loss-nya (0.5455 vs. 0.5462). Secara keseluruhan, RMSNorm memberikan dampak positif meskipun tidak signifikan, yang mengindikasikan bahwa normalisasi membantu proses optimasi berjalan sedikit lebih efisien.



Dari kurva loss, kedua model menunjukkan pola penurunan yang relatif serupa sepanjang 50 epoch, keduanya konvergen dengan stabil tanpa lonjakan yang mencolok. Namun, kurva milik model RMSNorm secara konsisten berada sedikit lebih rendah dibandingkan model tanpa normalisasi, baik pada training loss maupun validation loss. Ini menunjukkan bahwa RMSNorm membantu model belajar lebih efisien di setiap langkah pembaruan bobot.

Yang juga perlu diperhatikan adalah jarak antara training loss dan validation loss pada kedua model relatif kecil dan tidak melebar secara signifikan seiring bertambahnya epoch. Artinya, keduanya tidak mengalami overfitting yang berarti dalam rentang 50 epoch ini. RMSNorm dalam konteks ini bukan

sekadar mencegah overfitting, melainkan lebih berperan dalam mempercepat dan menstabilkan konvergensi.

Dari sisi distribusi bobot, kedua model menghasilkan pola yang cukup mirip mengingat keduanya dimulai dari inisialisasi yang sama. Namun pada model dengan RMSNorm, distribusi bobot cenderung lebih terpusat dan simetris di sekitar nol. Ini adalah efek yang diharapkan dari normalisasi, yaitu dengan menjaga skala aktivasi tetap terkontrol, bobot tidak perlu "bergerak terlalu jauh" untuk mengompensasi skala yang tidak seimbang antar layer.

Perbedaan yang lebih mencolok terlihat pada distribusi gradien. Model tanpa RMSNorm menunjukkan gradien yang lebih tersebar dan berpotensi tidak merata antar layer, terutama pada layer yang lebih dalam. Sementara itu, model dengan RMSNorm memiliki distribusi gradien yang lebih terkonsentrasi dan seragam di seluruh layer. Hal ini menunjukkan bahwa RMSNorm efektif dalam mengurangi risiko vanishing maupun exploding gradient, masalah umum pada jaringan yang lebih dalam, sehingga sinyal gradien mengalir lebih lancar dari output ke layer pertama selama backpropagation.

2.7 Perbandingan dengan Library SKLearn

Setelah diperoleh learning rate terbaik dari eksperimen 2.3 sebelumnya (yaitu 0.1), model FFNN dari scratch dibandingkan secara langsung dengan MLPClassifier dari library scikit-learn. Agar perbandingan adil, konfigurasi sklearn disetel semirip mungkin dengan FFNN buatan sendiri, yaitu dengan arsitektur satu hidden layer dengan 16 neuron, aktivasi ReLU, optimizer SGD, learning rate 0.1, batch size 32, dan 60 iterasi/epoch. Kedua model dilatih pada data training yang sama dan dievaluasi menggunakan metrik akurasi pada data validasi yang sama pula.

```
Final prediction comparison (FFNN vs sklearn)
FFNN (best LR=0.1) val_accuracy : 0.743500
sklearn MLPClassifier val_accuracy : 0.741500
Accuracy gap (sklearn - FFNN) : -0.002000
```

Hasil perbandingan menunjukkan bahwa performa kedua model hampir setara. FFNN yang dibangun dari scratch menghasilkan val accuracy sebesar 74.35%, sedangkan sklearn MLPClassifier menghasilkan 74.15% selisihnya hanya 0.2% dengan keunggulan tipis pada model scratch. Ini adalah hasil yang sangat baik, karena menunjukkan bahwa implementasi FFNN secara manual sudah cukup benar dan mampu mereproduksi perilaku yang mendekati library yang sudah teruji dan dioptimasi secara profesional.

Selisih kecil yang ada bisa disebabkan oleh perbedaan implementasi internal, misalnya cara sklearn menangani inisialisasi bobot, jadwal learning rate, atau detail numerik dalam proses backpropagation. Secara keseluruhan, kedua model menunjukkan kemampuan generalisasi yang serupa pada dataset ini, dan tidak ada tanda-tanda salah satunya mengalami overfitting yang jauh lebih parah dibanding yang lain.

BAB 3

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Implementasi Feedforward Neural Network (FFNN) pada tugas ini telah berhasil mencakup berbagai komponen penting, seperti fungsi aktivasi, loss function, inisialisasi bobot, regularisasi, serta optimizer (SGD dan Adam). Model mampu melakukan forward dan backward propagation dengan baik, ditunjukkan oleh penurunan nilai loss selama proses pelatihan.

Hasil eksperimen menunjukkan bahwa tidak semua peningkatan kompleksitas model meningkatkan performa. Penambahan depth justru menurunkan performa, sedangkan width tidak memberikan pengaruh signifikan. Fungsi aktivasi ReLU memberikan performa terbaik, sementara learning rate menjadi faktor yang sangat krusial dalam menentukan kecepatan dan stabilitas konvergensi.

Inisialisasi bobot memiliki pengaruh besar terhadap keberhasilan training, di mana metode zero gagal total, sedangkan uniform dan normal bekerja dengan baik. Regularisasi L2 terbukti lebih stabil dibandingkan L1, dan penggunaan RMSNorm membantu meningkatkan stabilitas training. Selain itu, Adam optimizer memberikan konvergensi yang lebih cepat dibandingkan SGD.

Perbandingan dengan scikit-learn menunjukkan bahwa implementasi dari scratch memiliki performa yang kompetitif, sehingga dapat disimpulkan bahwa model telah diimplementasikan dengan benar.

3.2 Saran

Pengembangan selanjutnya dapat dilakukan dengan menambahkan teknik seperti batch normalization, dropout, atau arsitektur yang lebih kompleks untuk meningkatkan performa pada jaringan yang lebih dalam. Selain itu, eksplorasi hyperparameter dan optimizer lain juga dapat dilakukan untuk memperoleh hasil yang lebih optimal.

Dari sisi implementasi, fitur seperti early stopping dan tuning otomatis dapat ditambahkan agar proses training menjadi lebih efisien dan adaptif.

PEMBAGIAN TUGAS

No.	NIM / Nama	Tugas
1.	13523095 / Rafif Farras	- Loss function - Data prep & preprocessing - Save load model - Weight Update - Eksperimen learning rate + perbandingan sklearn - Bonus RMSNorm - Bonus Autodiff
2.	13523101 / Barru Adi Utomo	- Fungsi aktivasi - Backward propagation - Regularisasi - Bonus 2 fungsi aktivasi - Eksperimen fungsi aktivasi - Eksperimen regularisasi - Bonus autodiff
3.	13523115 / Azfa Radhiyya Hakim	- Kelas FFNN - Inisialisasi bobot zero, uniform, dan normal - Forward propagation - Metode fit - Bonus Xavier dan He - Bonus Adam - Eksperimen depth width - Eksperimen inisialisasi bobot

REFERENSI

Kingma, D. P., & Ba, J. (2014). *Adam: A method for stochastic optimization*. arXiv:1412.6980. <https://arxiv.org/abs/1412.6980>

Powerpoint slide IF3270 Machine Learning

Laudani, A., et al. (2015). *On training efficiency of feedforward neural networks*.

LAMPIRAN

Tautan repository: <https://github.com/rafifrs/trizzzcuitt-MachineLearning-FFNN>

FORMAT DEKLARASI PENGGUNAAN AI SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama / NIM : Rafif Farras/13523095
Barru Adi Utomo/13523101
Azfa Radhiyya Hakim/13523115
Fakultas / Sekolah : STEI / Teknik Informatika
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas / Proposal : Tugas Besar 1 Pembelajaran Mesin

Menyatakan bahwa kami **menggunakan** AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Apabila menggunakan AI, berikut adalah rincian penggunaannya:

No	Lingkup Pekerjaan	Digunakan (Ya/Tidak)	Tingkat Penggunaan
1	Pemeriksaan Ejaan (Grammarly, Paperpal, DeepL, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, dll.)	Tidak	-
2	Pembuatan Teks (ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, dll.)	Ya	Rendah (untuk memperbaiki penyusunan kalimat)
3	Bantuan Diskusi dalam Penyusunan Konten (ChatGPT, Gemini, Copilot, Perplexity,	Ya	Rendah (untuk keperluan debugging)

	Jenni AI, Zoom Companion, dll.)		
4	Pembuatan Gambar, Video, dan/atau Grafik (Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, dll.)	Tidak	-

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan secara lengkap dan benar di dalam surat ini.

Bandung, 18 Maret 2026



Rafif Farras
13523095

Bandung, 18 Maret 2026



Barru Adi Utomo
13523101

Bandung, 18 Maret 2026



Azfa Radhiyya Hakim
13523115